PRACTICAL – 7

Implementation of making a change problem using dynamic programming.

Aim:

To implement the **Making Change Problem using Dynamic Programming** and determine the minimum numberofcoinsrequiredtomakeagivenamount.

Objective:

The objective of this experiment is to solve the coin change problem efficiently using dynamic programming by breaking it into smaller subproblems. The program calculates the minimum number of coins required to make a given amount from the available denominations and stores intermediate results to avoid repeated calculations.

Algorithm:

- Start.
- Read the number of available coin denominations n.
- Read the value of each coin.
- Read the target amount amount.
- Create an array dp of size amount + 1.
- Initialize all elements of dp with a large value (INF).
- Set dp[0] = 0.
- For each amount i from 1 to amount:

  For every coin denomination:

  If the coin value is less than or equal to i, update:

  $dp[i]=\min(dp[i],dp[i-coin]+1)$
- If dp[amount] is still INF, print that change cannot be made.
- Otherwise, print the minimum number of coins required.
- Stop.

Code:

```
# Making Change Problem using Dynamic Programming

def making_change(coins, amount):
    # dp[i] stores the minimum number of coins
    # required to make amount i
    dp = [float('inf')] * (amount + 1)

    # 0 coins are required to make amount 0
    dp[0] = 0
```

```
    # Calculate minimum coins for each amount
    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount]


# Take input from the user
n = int(input("Enter the number of coin denominations: "))

print("Enter the coin denominations:")
coins = []

for i in range(n):
    coins.append(int(input()))

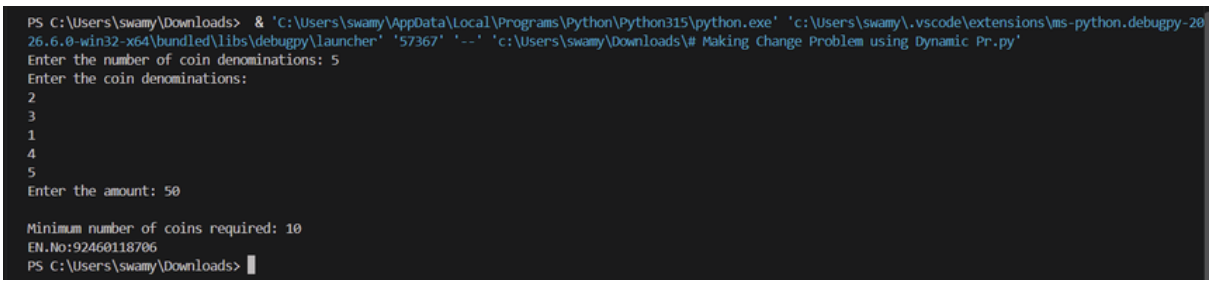
amount = int(input("Enter the amount: "))

# Find minimum number of coins
minimum_coins = making_change(coins, amount)

# Display result
if minimum_coins == float('inf'):
    print("Change cannot be made for the given amount.")
else:
    print("\nMinimum number of coins required:", minimum_coins)

# Enter your EN.No
print("EN.No:92460118706")
```

Output:



Time Complexity:

Case	Time Complexity	Description
Best Case	*O(n x A)*	Checks all n coins for each amount up to A.
Average Case	*O(n x A)*	The same DP table and coin checks are performed.
Worst case	*O(n x A)*	All amounts and all coin denominations are checked.

Space Complexity:

Space Complexity	Description
O(A)	A DP array of size A + 1 is used.